IT4010 Research Project – 2025 July

Topic Assessment Form

Project ID : 25-26J-455

1. Topic (12 words max)

Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	**25-26J-455**
Year	**2025**

5. Brief description of the research problem including references (200 – 500 words max) – references not included in word count.

Large enterprises still depend on decades-old mainframes and monolithic applications for mission-critical workflows such as payment clearing, inventory management, and citizen services. Although generally stable, these legacy systems are plagued by three systemic challenges: (i) rigid, proprietary interfaces that hinder cloud or mobile integration; (ii) high maintenance costs due to dwindling expertise in obsolete technologies like COBOL, Visual Basic, and PowerBuilder; and (iii) operational blind spots that prevent real-time analytics, monitoring, and timely security updates [1], [2]. According to Gartner, nearly 70% of digital transformation budgets are spent simply on maintenance rather than innovation, highlighting the severity of the problem [3].

Traditional modernization strategies—such as re-hosting, re-platforming, or complete rewriting—are expensive, disruptive, and often introduce significant business risks due to system downtime and data migration challenges [1]. These approaches typically require a full understanding of the legacy codebase, which is rarely documented or modular.

Recent advancements in **agentic AI**—notably in systems like **Self-Refine** [4] and **Voyager** [5]—demonstrate that large language model (LLM)-based agents with memory, goal-driven behavior, and self-evaluation capabilities can reason and act autonomously in complex environments. These systems have shown promise in exploration, tool use, and task refinement without constant human supervision. However, current research lacks practical implementations of such agentic frameworks in the context of **middleware for live legacy systems**, where the goal is not to replace the system but to augment and extend its usability.

This gap raises a critical question: **Can a self-improving, agentic AI middleware wrap around an unchanged legacy application to provide modern APIs, analytics, and security capabilities without altering the core code?** This research aims to answer that question by designing an AI-driven middleware layer that reduces modernization lead time, mitigates technical debt, and enables incremental evolution toward cloud-native architectures.

References:

[1] M. Bisbal et al., "Legacy information systems: Issues and directions," *IEEE Software*, vol. 16, no. 5, pp. 103–111, 1999.

[2] K. Bennett, "Legacy systems: Coping with success," *IEEE Software*, vol. 12, no. 1, pp. 19–23, 1995.

[3] Gartner, "Modernization Strategies for Legacy Systems," *Gartner Research*, 2024.

[4] Y. Mialon et al., "Self-Refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[5] J. Xu et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models," *arXiv preprint arXiv:2305.16291*, 2024.

[6] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, 2nd ed., Cambridge University Press, 2009.

6. Brief description of the nature of the solution including a conceptual diagram (250 words max)

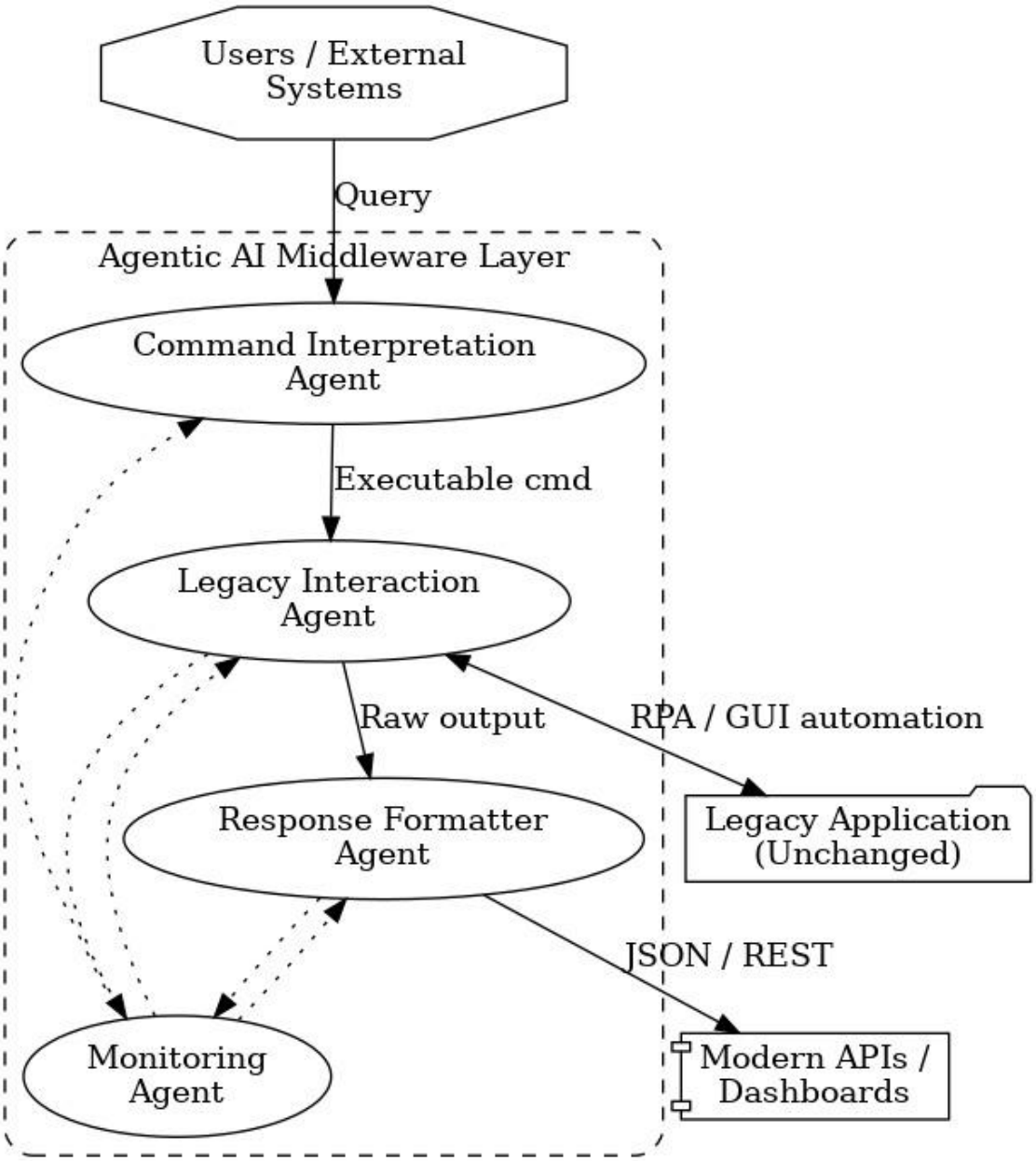
This research proposes the development of an **Agentic AI Middleware Layer** that acts as an intelligent interface between users (or external systems) and legacy applications—without altering the core legacy codebase. The middleware leverages **agentic AI capabilities**, including planning, memory, self-critique, and natural language understanding, to interpret user intents and convert them into executable actions on the legacy system.

The middleware will comprise multiple modular agents:

- A **Command Interpretation Agent** to understand user queries (e.g., "Get patient admission records from last week").
- A **Legacy Interaction Agent** that uses robotic process automation (RPA), scripting, or GUI simulation to execute commands on the legacy system.
- A **Response Formatter Agent** to translate outputs into modern formats such as JSON or REST APIs.
- A **Monitoring Agent** to track performance, suggest optimizations, and handle exceptions.

By abstracting the legacy system's complexity, the middleware provides **modern APIs**, dashboards, and analytics without invasive code changes. It can also integrate with external systems (e.g., cloud storage, mobile apps) while continuing to learn and optimize over time.

This solution reduces modernization lead-time, enables real-time interaction with legacy systems, and minimizes risk by operating non-intrusively. It serves as a **transitional architecture**—supporting organizations until full system replacement becomes feasible.



7. Brief description of specialized domain expertise, knowledge, and data requirements (300 words max)

The proposed research sits at the intersection of **software engineering**, **artificial intelligence**, and **enterprise systems integration**. It requires domain expertise in the following areas:

1. **Legacy Systems Architecture:**
   Understanding of mainframe and monolithic system structures (e.g., COBOL-based financial systems, Visual Basic hospital applications) is essential. This includes their data access patterns, UI workflows, and system constraints.
2. **Agentic AI and Large Language Models (LLMs):**
   Proficiency in deploying and fine-tuning LLMs (e.g., GPT, Claude) and frameworks like Self-Refine or AutoGPT is required to build autonomous agents capable of reasoning, self-correction, and planning within dynamic software environments.
3. **Middleware and API Design:**
   Knowledge of middleware architecture, RESTful APIs, and service-oriented design is crucial for enabling interaction between the AI agents and both legacy systems and modern interfaces.
4. **Robotic Process Automation (RPA):**
   Familiarity with RPA tools (e.g., UiPath, Automation Anywhere) or custom script-based automation is needed to simulate interactions with GUI-only legacy systems where no APIs are available.
5. **Security and Compliance:**
   Since many legacy systems operate in sensitive domains (e.g., healthcare, finance), expertise in secure data handling, access control, and regulatory compliance (e.g., HIPAA, GDPR) is necessary.

Data Requirements:

- Sample datasets or anonymized user logs from legacy systems (with permission).
- Access to sandboxed versions of legacy interfaces for testing agent behavior.
- Pretrained LLMs and open-source agent frameworks for customization.

8. Objectives and Novelty

Main Objective Design and implement a self-improving agentic middleware that modernizes legacy systems *without modifying their source code*.			
Member Name with Registration No	Sub Objective	Tasks	Novelty
It22355850 M Z M Ayyash	Develop a modular agentic middleware core architecture	1. Design a plug-and-play AI agent management layer 2. Implement agent orchestration using event-driven triggers 3. Build modular legacy interaction pipelines 4. Enable auto-scaling of agents based on task complexity	This component introduces a reusable and modular AI middleware framework capable of dynamically orchestrating intelligent agents based on task needs. It allows for seamless expansion and reusability without altering the legacy system's core.
It22355478 M A M MUFTHI	Build natural language to system task translator using LLM agents	1. Develop an intent recognition module with self-correction 2. Implement task planning using LLM chains 3. Auto-generate system commands from NL input 4. Build feedback-driven prompt refinement loop	This component enables large language model (LLM) agents to interpret human-like queries and translate them into structured, legacy-compatible system tasks—autonomously improving their translation accuracy through iterative feedback and self-correction.

It22331472 S M A C M RASHAD	Create a low-code RPA connector layer for legacy systems	1. Develop AI-driven screen and form recognizer for old GUIs 2. Implement RPA agents with adaptive macro recording 3. Build dynamic GUI simulators 4. Expose legacy actions as REST/GraphQL APIs	This component transforms GUI-only legacy systems into programmable interfaces by leveraging AI-enhanced RPA techniques, allowing for the exposure of legacy operations as modern APIs without requiring source code access or modification.
It22311436 D N Rajapaksha	Implement self-optimizing agent coordination and response system	1. Develop agent-to-agent communication protocols 2. Build self-monitoring agent state tracker 3. Implement decision trees to optimize agent delegation 4. Enable continuous learning from historical interactions	This component introduces an intelligent coordination framework where agents communicate, delegate, and self-optimize using learned behavior over time—enhancing task performance autonomously without external reconfiguration or human intervention.

9. Individual component description of how it is complied with the specialization.

Member Name with Registration No	Description
It22355850 M Z M Ayyash	Designs the middleware core architecture leveraging expertise in software architecture and systems engineering, ensuring modularity and scalability.
It22355478 M A M MUFTHI	Develops natural language processing agents using knowledge in AI, machine learning, and large language models to translate human intent effectively.
It22331472 S M A C M RASHAD	Builds RPA connectors applying skills in automation, legacy system integration, and scripting to bridge UI-only legacy systems with modern interfaces.
It22311436 D N Rajapaksha	Implements self-optimizing agent coordination algorithms based on background in AI multi-agent systems and optimization techniques.

10. Supervisor details

	Title	First Name	Last Name	Signature
Supervisor	Mr	Vishan	Jayasinghearachchi	26/06/2025
Co-Supervisor	Dr	Dharshana	Kasthurirathne	26/06/25
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				

This part is to be filled by the Topic Screening Staff members.

a) Does the chosen research topic possess a comprehensive scope suitable for a final-year project?

Yes		No	

b) Does the proposed topic exhibit novelty?

Yes		No	

c) Do you believe they have the capability to successfully execute the proposed project?

Yes		No	

d) Do the proposed sub-objectives reflect the students' areas of specialization?

Yes		No	

e) Supervisor's Evaluation and Recommendation for the Research topic:

IT4010 Research Project – 2025 July Topic Assessment Form

Acceptable: Mark/Select as necessary

Topic Assessment Accepted	
Topic Assessment Accepted with minor changes*	
Topic Assessment to be Resubmitted with major changes*	
Topic Assessment Rejected. Topic must be changed	

* Detailed comments given below

Comments

Staff Member's Name	Signature

*__Important__:

1. According to the comments given by the evaluator, make the necessary modifications and get the approval by the **Evaluator**.
2. If the project topic is rejected, identify a new topic, and request the RP Team for a new topic assessment.